

Student 1

Register Number 192311278

Student Name AKSHARAA BALASUBRMANIAN

Q1. Job Scheduling (Backtracking / NP-Hard Problem)

A company must assign jobs to employees such that each job is completed within a deadline and no employee handles more than one job at a time. Clearly define the problem and identify constraints related to time, resource availability, and job requirements. Analyze how these constraints interact and prioritize them logically. Develop a feasible solution using backtracking or constraint satisfaction techniques. Apply relevant concepts such as pruning and recursive assignment. Justify how your solution satisfies all constraints efficiently.

1. Problem Definition

Job scheduling is the process of assigning a set of jobs to available employees so that every job is completed within its specified deadline while satisfying employee availability and job-specific requirements.

In this problem, each job has:

- A processing time
- A deadline
- A required skill
- A priority

Each employee has:

- An availability period
- A specific skill set
- A workload capacity

The objective is to find a **feasible schedule** in which every job is assigned to a suitable employee and completed before its deadline, while ensuring that an employee does not perform two jobs simultaneously.

A **backtracking approach** is used to assign jobs recursively. Whenever a partial assignment violates a constraint, that branch is immediately rejected using **pruning**.

2. Objective

The main objectives are:

1. Assign every job to an available employee.
2. Ensure that the employee has the required skill.
3. Ensure that each job finishes before its deadline.
4. Prevent overlapping jobs for the same employee.
5. Respect employee working hours and resource availability.
6. Use backtracking to explore feasible assignments.
7. Prune invalid assignments as early as possible.
8. Produce a valid schedule if one exists.

3. Inputs

The algorithm takes the following inputs:

Jobs

Each job contains:

	Job	Processing Time	Deadline	Required Skill	Priority
--	-----	-----------------	----------	----------------	----------

J1	2 hours	4	Java	High
J2	1 hour	3	Python	Medium
J3	2 hours	6	Database	High
J4	1 hour	5	Java	Low

Employees

	Employee	Available Time	Skills
--	----------	----------------	--------

E1	8 hours	Java, Database
E2	8 hours	Python, Java
E3	6 hours	Database

4. Expected Output

The output should contain a valid assignment such as:

Job	Employee	Start Time	End Time	Deadline	Status
J1	E1	0	2	4	Completed
J2	E2	0	1	3	Completed
J3	E3	0	2	6	Completed
J4	E1	2	3	5	Completed

The schedule is valid because:

- Every job is assigned.
- Employees possess the required skills.
- No employee handles two jobs at the same time.
- All jobs finish before their deadlines.
- Employee availability is not exceeded.

5. Constraints

The scheduling problem has three major categories of constraints.

A. Time Constraints

Every job must be completed before its deadline.

For job J_i :

$$\text{CompletionTime}(J_i) \leq \text{Deadline}(J_i)$$

For example:

J1:

Start = 0

Duration = 2

Completion = 2

Deadline = 4

$$2 \leq 4 \rightarrow \text{Valid}$$

B. Resource Availability Constraints

An employee cannot perform two jobs simultaneously.

For example, if employee E1 is working on:

J1: 0 ——— 2

another job assigned to E1 cannot start before time 2.

Invalid:

J1: 0 ——— 2

J4: 1 ——— 2

Valid:

J1: 0 ——— 2

J4: 2 ——— 3

C. Job Requirement Constraints

An employee can perform a job only if they possess the required skill.

For example:

J1 requires Java

If:

E1 = Java + Database

then:

$E1 \rightarrow J1 = \text{Valid}$

But if:

E3 = Database

then:

$E3 \rightarrow J1 = \text{Invalid}$

6. Interaction Between Constraints

The constraints are interdependent.

For example, an employee may have the correct skill but may not be available before the job's deadline.

Similarly, an employee may be available but may not have the required skill.

Therefore, an assignment is accepted only when **all constraints are satisfied simultaneously**.

Job		

Skill	Time	Priority

Employee Available?		
Yes / No		
Check Time Slot		

Valid	Invalid	
Assign Job		

7. Constraint Priority

The constraints can be checked in the following logical order:

Priority 1 – Job Requirement

First check whether the employee has the required skill.

RequiredSkill $\in$ EmployeeSkills

If not, immediately reject the assignment.

Priority 2 – Employee Availability

Check whether the employee is available during the required time.

Priority 3 – Deadline

Check whether:

StartTime + ProcessingTime $\leq$ Deadline

Priority 4 – Resource Conflict

Check whether the employee is already assigned another job during that time.

Priority 5 – Optimization

After satisfying all mandatory constraints, high-priority jobs can be scheduled earlier.

This ordering improves pruning because impossible employee-job combinations are eliminated early.

8. Backtracking Strategy

The algorithm assigns jobs **one at a time**.

For every job:

1. Select an employee.
2. Check whether the employee has the required skill.
3. Find an available time slot.
4. Check the deadline.
5. Check for overlapping jobs.
6. If all conditions are satisfied, assign the job.
7. Recursively schedule the next job.
8. If the next assignment fails, remove the current assignment.
9. Try another employee or time slot.

This process continues until all jobs are scheduled.

9. Pseudocode

Main Algorithm

ALGORITHM JOB_SCHEDULING_BACKTRACKING

INPUT:

Jobs[1...N]

Each job contains:

- ProcessingTime
- Deadline
- RequiredSkill
- Priority

Employees[1...M]

Each employee contains:

- AvailableTime
- Skills

OUTPUT:

A feasible schedule containing employee,
start time and end time for every job

BEGIN

Sort Jobs according to priority
and earliest deadline

Initialize Schedule as EMPTY

IF BACKTRACK(1) = TRUE THEN

PRINT "Feasible Schedule Found"

FOR each job in Schedule DO

PRINT Job ID

PRINT Assigned Employee

PRINT Start Time

PRINT End Time

END FOR

ELSE

PRINT "No feasible schedule exists"

END IF

END

10. Recursive Backtracking Procedure

PROCEDURE BACKTRACK(jobIndex)

BEGIN

IF jobIndex > N THEN

RETURN TRUE

END IF


```
job ← Jobs[jobIndex]
```

```
FOR each employee IN Employees DO
```

```
    // Constraint 1: Skill requirement
```

```
    IF job.RequiredSkill NOT IN employee.Skills THEN
```

```
        CONTINUE
```

```
    END IF
```

```
    // Try possible start times
```

```
    FOR startTime ← 0 TO employee.AvailableTime DO
```

```
        endTime ← startTime + job.ProcessingTime
```

```
        // Constraint 2: Deadline
```

```
        IF endTime > job.Deadline THEN
```

```
            BREAK
```

```
        END IF
```

```
// Constraint 3: Employee availability  
IF endTime > employee.AvailableTime THEN
```

```
    BREAK
```

```
END IF
```

```
// Constraint 4: No overlapping jobs
```

```
IF HAS_CONFLICT(employee, startTime, endTime) = TRUE THEN
```

```
    CONTINUE
```

```
END IF
```

```
// Assign the job
```

```
Schedule[jobIndex] ←
```

```
    (job, employee, startTime, endTime)
```

```
// Recursive assignment
```

```
IF BACKTRACK(jobIndex + 1) = TRUE THEN
```

```
    RETURN TRUE
```

```
END IF
```

```
// Backtrack
```

```
REMOVE Schedule[jobIndex]
```

```
END FOR
```

```
END FOR
```

```
RETURN FALSE
```

```
END
```

11. Conflict Checking

The following procedure checks whether the employee is already performing another job during the selected time.

```
FUNCTION HAS_CONFLICT(employee, startTime, endTime)
```

```
BEGIN
```

```
FOR each scheduledJob IN Schedule DO
```

```
IF scheduledJob.employee = employee THEN
```

```
IF startTime < scheduledJob.endTime AND
```

```
endTime > scheduledJob.startTime THEN
```

RETURN TRUE

END IF

END IF

END FOR

RETURN FALSE

END

The condition:

$\text{startTime} < \text{existingEnd}$

AND

$\text{endTime} > \text{existingStart}$

detects overlapping jobs.

12. Pruning Strategy

Pruning is one of the most important parts of the algorithm.

Instead of continuing with an invalid assignment, the algorithm immediately eliminates it.

Pruning Rule 1 – Skill Mismatch

IF RequiredSkill NOT IN EmployeeSkills

→ Reject assignment

Pruning Rule 2 – Deadline Violation

IF $\text{StartTime} + \text{ProcessingTime} > \text{Deadline}$

→ Reject assignment

Pruning Rule 3 – Employee Capacity

IF $\text{EndTime} > \text{EmployeeAvailableTime}$

→ Reject assignment

Pruning Rule 4 – Time Conflict

IF Employee already has an overlapping job

→ Reject assignment

Pruning Rule 5 – Remaining Capacity

If the remaining available employee time is insufficient to complete the remaining jobs, the branch can also be discarded.

IF $\text{RemainingEmployeeCapacity} < \text{RemainingJobTime}$

→ Backtrack immediately

This avoids exploring impossible branches.

13. Example Execution

Consider:

Jobs:

J1 → 2 hours, deadline 4, Java

J2 → 1 hour, deadline 3, Python

J3 → 2 hours, deadline 6, Database

J4 → 1 hour, deadline 5, Java

Employees:

E1 → Java, Database

E2 → Python, Java

E3 → Database

Step 1 – Schedule J1

J1 requires Java.

Possible employees:

E1 → Java ✓

E2 → Java ✓

E3 → Java ✗

Suppose:

J1 → E1

Start = 0

End = 2

Step 2 – Schedule J2

J2 requires Python.

E1 → Python ✗

E2 → Python ✓

E3 → Python ✗

Therefore:

J2 → E2

Start = 0

End = 1

Step 3 – Schedule J3

J3 requires Database.

E1 → Database ✓

E2 → Database ✗

E3 → Database ✓

Suppose:

J3 → E3

Start = 0

End = 2

Step 4 – Schedule J4

J4 requires Java.

E1 → Java ✓

E2 → Java ✓

E3 → Java X

If assigned to E1:

J1 → E1: 0–2

J4 → E1: 2–3

There is no overlap.

Therefore:

J4 → E1

Start = 2

End = 3

14. Final Feasible Schedule

Job	Employee	Start	End	Deadline	Result
-----	----------	-------	-----	----------	--------

J1	E1	0	2	4	✓ Valid
----	----	---	---	---	---------

J2	E2	0	1	3	✓ Valid
----	----	---	---	---	---------

J3	E3	0	2	6	✓ Valid
----	----	---	---	---	---------

J4	E1	2	3	5	✓ Valid
----	----	---	---	---	---------

Therefore, **all four jobs are successfully scheduled.**

15. Constraint Verification

Time Constraint

J1: $2 \leq 4$ ✓

J2: $1 \leq 3$ ✓

J3: $2 \leq 6$ ✓

J4: $3 \leq 5$ ✓

All jobs meet their deadlines.

Skill Constraint

J1 → E1 → Java ✓

J2 → E2 → Python ✓

J3 → E3 → Database ✓

J4 → E1 → Java ✓

All employees have the required skills.

Resource Constraint

Employee E1:

J1: 0 — 2

J4: 2 — 3

No overlap occurs.

Employee E2:

J2: 0 — 1

Employee E3:

J3: 0 — 2

Therefore, no employee handles more than one job simultaneously.

16. Backtracking Flow

Start



Select Next Job



Select an Employee



Check Required Skill



No Yes



Prune Find Time Slot



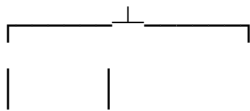
Check Deadline



No Yes



Prune Check Conflict



Conflict No Conflict



Prune Assign Job



Schedule Next Job



All Jobs Done?



Yes No



Solution Continue

If a later job cannot be assigned:

Failed Assignment

↓

Remove Previous Job

↓

Try Another Employee

↓

Try Another Time Slot

↓

Continue Search

17. Why Backtracking is Suitable

Backtracking is suitable because the scheduling problem involves multiple interacting constraints.

A simple greedy approach might assign the first available employee to a job without considering future jobs.

For example:

J1 → E1

J2 → E2

J3 → E1

may appear valid initially, but a later job may require the only employee with a particular skill.

Backtracking solves this problem by exploring alternatives:

Assignment

↓

Check constraints

↓

Feasible?

/ \

Yes No

↓ ↓

Continue Prune

↓

Future assignment fails?

↓

Backtrack

↓

Try another assignment

Thus, the algorithm can recover from an earlier poor assignment.

18. Complexity Analysis

Suppose there are:

- N jobs
- M employees
- T possible time slots

In the worst case, the algorithm may explore approximately:

$$O((M \times T)^N)$$

possible assignments.

This is exponential in nature, which is expected for difficult scheduling/constraint problems.

However, **pruning significantly reduces the practical search space** by eliminating:

- Skill-incompatible employees
- Deadline violations
- Resource conflicts
- Availability violations
- Impossible remaining schedules

19. Advantages

- Handles multiple constraints simultaneously.

- Finds a feasible solution systematically.
- Automatically backtracks from invalid decisions.
- Prunes impossible branches early.
- Can handle different employee skills.
- Considers deadlines and availability.
- Can be extended to optimize priorities and workload.

20. Limitations

- Worst-case execution time can be high.
- The number of possible assignments increases rapidly with the number of jobs and employees.
- Basic backtracking may become slow for large scheduling problems.
- Additional optimization techniques may be required for industrial-scale scheduling.

21. Possible Optimizations

The algorithm can be improved by:

1. Schedule urgent jobs first

Sort jobs by earliest deadline.

Earliest Deadline First

2. Schedule difficult jobs first

Assign jobs requiring rare skills before jobs that many employees can perform.

3. Forward Checking

After assigning a job, check whether all remaining jobs still have at least one possible employee.

4. Remaining Capacity Check

If available employee time is insufficient for remaining jobs:

Prune branch

5. Priority-Based Assignment

High-priority jobs can be considered before low-priority jobs.

These optimizations reduce unnecessary exploration.

22. Conclusion

The **Job Scheduling Problem** can be effectively modeled as a **constraint satisfaction problem** and solved using **recursive backtracking**. The algorithm considers job requirements, employee skills, time availability, deadlines, and resource conflicts before making an assignment. If an assignment violates any constraint, it is immediately **pruned**, preventing unnecessary exploration. If a later assignment becomes impossible, the algorithm backtracks and tries an alternative employee or time slot.

Thus, the proposed solution provides a **systematic and logically correct method for obtaining a feasible job schedule**, while pruning and constraint prioritization improve its efficiency.